

Wireshark Packet Analysis – Report

1. Title:

Wireshark Packet Analysis

2. Objective:

The objective of this task is to capture and analyze live network traffic using Wireshark and understand the behavior of different network protocols such as TCP, HTTP, and DNS. The task also involves identifying packet flow and checking for any abnormal or malformed packets.

3. Task Description:

In this task, I was assigned to perform network packet analysis using Wireshark. The goal was to capture real-time network traffic, apply protocol filters, and analyze communication between devices. This helps in understanding how data is transmitted over a network and how different protocols operate during communication.

4. Tools Used:

- Wireshark Network Analyzer
- Operating System (Windows/Linux)
- Active Internet Connection (Wi-Fi/Ethernet)

5. Methodology / Implementation:

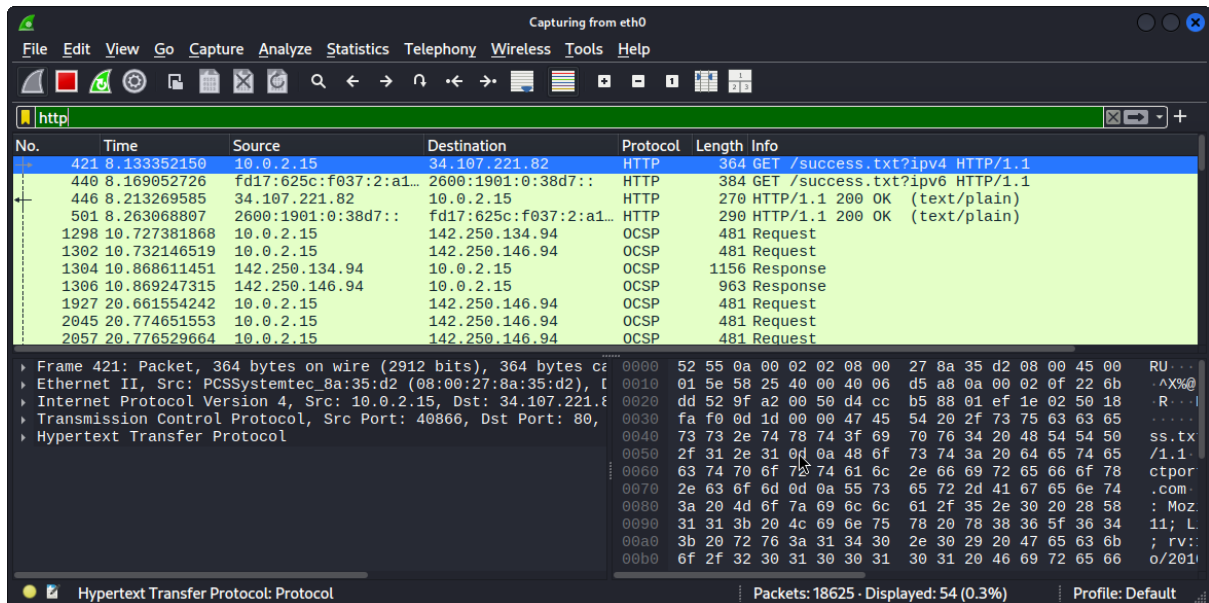
- Wireshark application was opened and a live network interface was selected for packet capture.
- Packet capture was started to monitor real-time network traffic.
- Different protocol filters such as **TCP, HTTP, and DNS** were applied to analyze specific traffic types.
- TCP stream was analyzed to understand the **three-way handshake process (SYN, SYN-ACK, ACK)**.
- DNS queries and HTTP requests were observed to understand request-response communication.
- Packet details were inspected to understand source, destination, protocol behavior, and data flow.
- A check was performed for malformed or abnormal packets using Wireshark filters.

6. Observations / Analysis:

- Multiple network packets were captured successfully during the session.
- TCP protocol communication clearly showed the three-way handshake process.
- DNS packets were observed, indicating domain name resolution activities.
- HTTP packets showed request and response communication between client and server.
- No critical abnormalities were observed in normal packet flow.
- Malformed packet filter was checked and no significant malformed packets were detected during the analysis.

7. Screenshots / Evidence:

HTTP:



Wireshark interface showing HTTP traffic capture on eth0. The packet list displays several HTTP requests and responses. The packet details pane shows the structure of a selected packet.

No.	Time	Source	Destination	Protocol	Length	Info
421	8.133352150	10.0.2.15	34.107.221.82	HTTP	364	GET /success.txt?ipv4 HTTP/1.1
440	8.169052726	fd17:625c:f037:2:a1...	2600:1901:0:38d7::	HTTP	384	GET /success.txt?ipv6 HTTP/1.1
446	8.213269585	34.107.221.82	10.0.2.15	HTTP	270	HTTP/1.1 200 OK (text/plain)
501	8.263068807	2600:1901:0:38d7::	fd17:625c:f037:2:a1...	HTTP	290	HTTP/1.1 200 OK (text/plain)
1298	10.727381868	10.0.2.15	142.250.134.94	OCSP	481	Request
1302	10.732146519	10.0.2.15	142.250.146.94	OCSP	481	Request
1304	10.868611451	142.250.134.94	10.0.2.15	OCSP	1156	Response
1306	10.869247315	142.250.146.94	10.0.2.15	OCSP	963	Response
1927	20.661554242	10.0.2.15	142.250.146.94	OCSP	481	Request
2045	20.774651553	10.0.2.15	142.250.146.94	OCSP	481	Request
2057	20.776529664	10.0.2.15	142.250.146.94	OCSP	481	Request

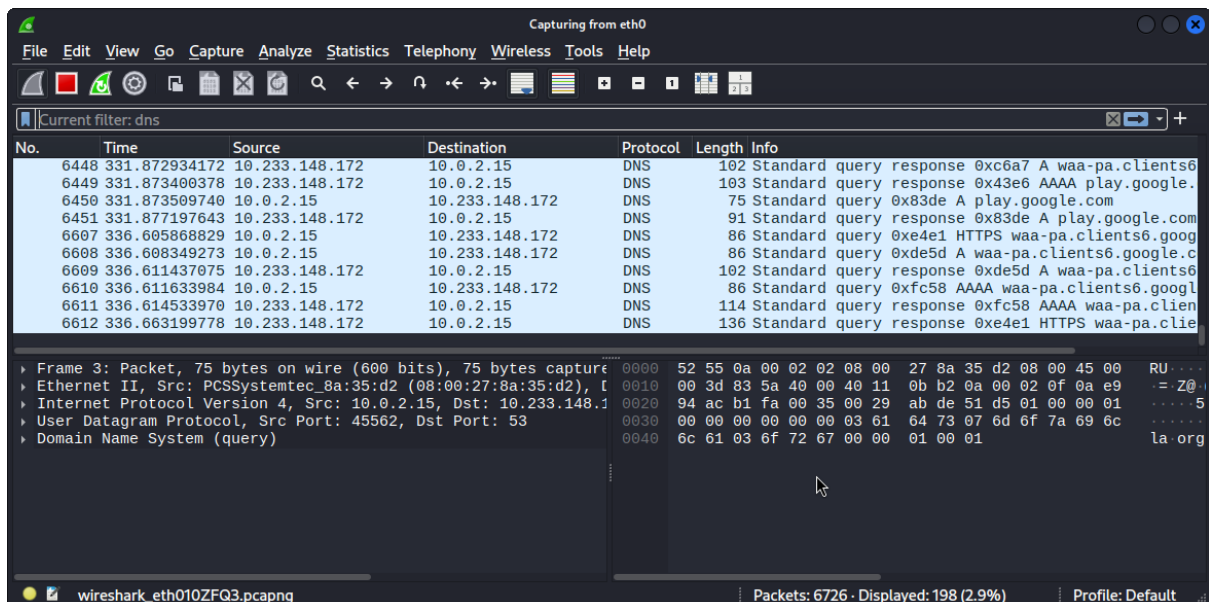
Frame 421: Packet, 364 bytes on wire (2912 bits), 364 bytes captured (2912 bits) on interface eth0, capture length 2912 bytes

- Ethernet II, Src: PCSSystemtec_8a:35:d2 (08:00:27:8a:35:d2), Dst: 34.107.221.82 (01:00:00:00:00:00)
- Internet Protocol Version 4, Src: 10.0.2.15, Dst: 34.107.221.82
- Transmission Control Protocol, Src Port: 40866, Dst Port: 80
- Hypertext Transfer Protocol

Hypertext Transfer Protocol: Protocol

Packets: 18625 - Displayed: 54 (0.3%) Profile: Default

DNS:



Wireshark interface showing DNS traffic capture on eth0. The packet list displays several DNS queries and responses. The packet details pane shows the structure of a selected packet.

No.	Time	Source	Destination	Protocol	Length	Info
6448	331.872934172	10.233.148.172	10.0.2.15	DNS	102	Standard query response 0xc6a7 A waa-pa.clients6
6449	331.873400378	10.233.148.172	10.0.2.15	DNS	103	Standard query response 0x43e6 AAAA play.google.
6450	331.873509740	10.0.2.15	10.233.148.172	DNS	75	Standard query 0x83de A play.google.com
6451	331.877197643	10.233.148.172	10.0.2.15	DNS	91	Standard query response 0x83de A play.google.com
6607	336.605868829	10.0.2.15	10.233.148.172	DNS	86	Standard query 0xe4e1 HTTPS waa-pa.clients6.goog
6608	336.608349273	10.0.2.15	10.233.148.172	DNS	86	Standard query 0xde5d A waa-pa.clients6.google.c
6609	336.611437075	10.233.148.172	10.0.2.15	DNS	102	Standard query response 0xde5d A waa-pa.clients6
6610	336.611633984	10.0.2.15	10.233.148.172	DNS	86	Standard query 0xfc58 AAAA waa-pa.clients6.googl
6611	336.614533970	10.233.148.172	10.0.2.15	DNS	114	Standard query response 0xfc58 AAAA waa-pa.clie
6612	336.663199778	10.233.148.172	10.0.2.15	DNS	136	Standard query response 0xe4e1 HTTPS waa-pa.clie

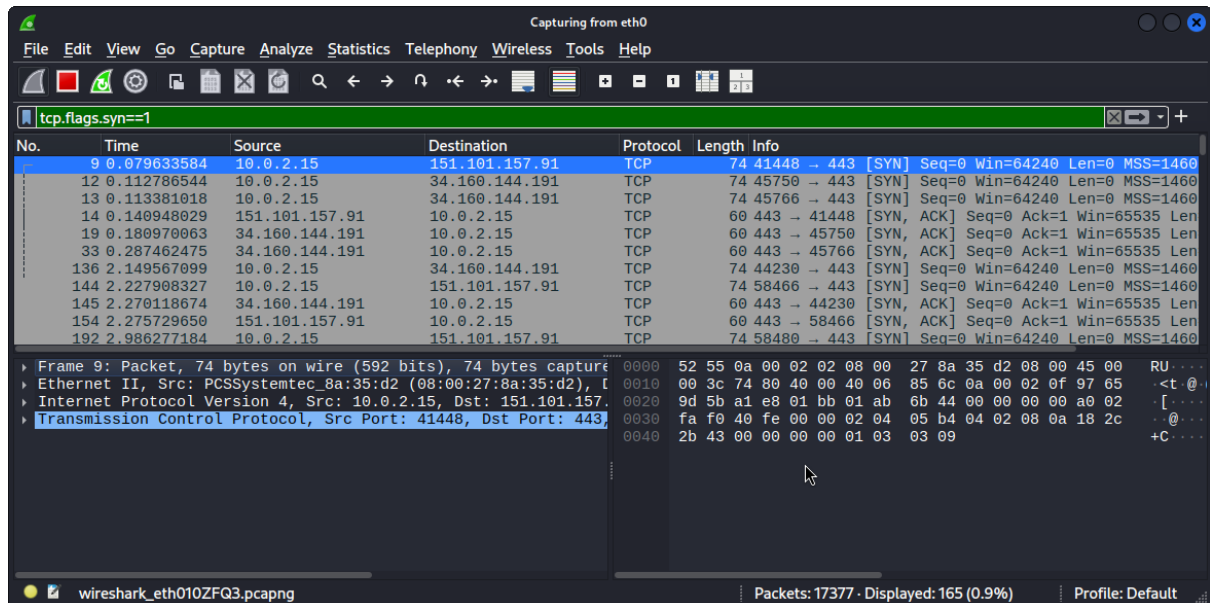
Frame 3: Packet, 75 bytes on wire (600 bits), 75 bytes captured (600 bits) on interface eth0, capture length 600 bytes

- Ethernet II, Src: PCSSystemtec_8a:35:d2 (08:00:27:8a:35:d2), Dst: 10.233.148.172 (01:00:00:00:00:00)
- Internet Protocol Version 4, Src: 10.0.2.15, Dst: 10.233.148.172
- User Datagram Protocol, Src Port: 45562, Dst Port: 53
- Domain Name System (query)

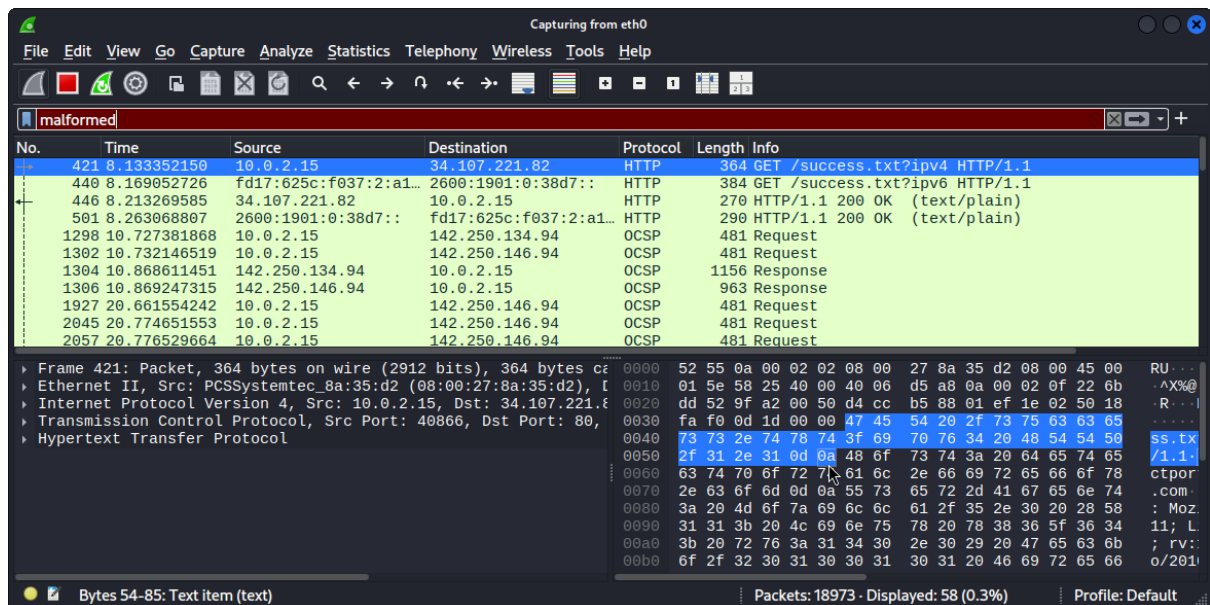
wireshark_eth010ZFQ3.pcapng

Packets: 6726 - Displayed: 198 (2.9%) Profile: Default

TCP Handshakes:



Malformed:



Example structure:

- **Figure 1:** HTTP
- **Figure 2:** DNS
- **Figure 3:** TCP three-way handshake analysis
- **Figure 4:** Malformed result (if applied)

8. Results / Findings:

The network traffic was successfully captured and analyzed using Wireshark. Different protocols such as TCP, HTTP, and DNS were identified and studied. The TCP connection establishment process was clearly observed through the three-way handshake. The packet flow between source and destination was understood effectively. No malformed or suspicious packets were detected during the analysis.

9. Conclusion:

This task provided practical exposure to network packet analysis using Wireshark. It helped in understanding how real-time network communication takes place and how different protocols operate in a network environment. The exercise improved knowledge of packet structure, protocol behavior, and traffic analysis techniques.